

Flutter Fundamentals –

UI – Widgets and Layouts

Contents

1.	Widgets.....	2
1.1.	Design systems.....	2
1.2.	Base widgets.....	3
2.	Layouts.....	6
2.1.	Overview.....	6
2.2.	Conceptual example.....	6
	Bài tập 1. Vẽ widget tree.....	6
2.3.	Layout a widget.....	7
	Bài tập 2. Bố cục cho một widget.....	7
2.4.	Lay out multiple widgets vertically and horizontally.....	8
	Bài tập 3. Vẽ cây widget.....	8
	Bài tập 4. Aligning widgets.....	9
	Bài tập 5. Sizing widgets.....	10
	Bài tập 6. Packing widgets.....	10
	Bài tập 7. Nesting rows and columns.....	10
2.5.	Common layout widgets.....	11
2.6.	Constraints.....	11
3.	Thực hành.....	12
	Bài tập 8. Co giãn chiều cao Column theo con.....	12
	Bài tập 9. Tạo layout tổng hợp.....	15

1. Widgets

1.1. Design systems

Tham khảo: <https://docs.flutter.dev/ui/widgets>

Flutter ships with two design systems as part of the SDK.

Cupertino: <https://docs.flutter.dev/ui/widgets/cupertino>

Beautiful and high-fidelity widgets that align with Apple's Human Interface Guidelines for iOS and macOS.

Material design: <https://docs.flutter.dev/ui/widgets/material>

Visual, behavioral, and motion-rich widgets implementing the Material 3 design specification.

You can find many more design systems created by the Flutter community on pub.dev, the package repository for Dart and Flutter. For example, the Windows-inspired `fluent_ui`, macOS-inspired `macos_ui`, and the Ubuntu-inspired `yaru` widgets.

Dịch:

Cupertino

Các widget đẹp mắt, có độ trung thực cao và tuân thủ Nguyên tắc Giao diện Người dùng (Human Interface Guidelines) của Apple dành cho iOS và macOS.

Material Design

Các widget phong phú về hình ảnh, hành vi và chuyển động, được xây dựng dựa trên tiêu chuẩn thiết kế Material 3.

Bạn có thể tìm thấy nhiều hệ thống thiết kế khác do cộng đồng Flutter tạo ra trên pub.dev – kho lưu trữ gói dành cho Dart và Flutter. Ví dụ như ``fluent_ui`` (lấy cảm hứng từ Windows), ``macos_ui`` (lấy cảm hứng từ macOS) và các widget ``yaru`` (lấy cảm hứng từ Ubuntu).

1.2. Base widgets

Tham khảo: <https://docs.flutter.dev/ui/widgets>

Những widgets cơ sở:

Base widgets

Base widgets support a range of common rendering options like input, layout, and text.

Accessibility

Make your app accessible.

Animation and motion

Bring animations to your app.

Assets, images, and icons

Manage assets, display images, and show icons.

Async

Widgets supporting async patterns in your Flutter apps.

Basics

Widgets to know before building your first Flutter app.

Input

Take user input in addition to input widgets in Material components and Cupertino.

Interaction models

Respond to touch events and route users to different views.

Layout

Arrange other widgets columns, rows, grids, and many other layouts.

Painting and effects

These widgets apply visual effects to the children without changing their layout, size, or position.

Scrolling

Scroll multiple widgets as children of the parent.

Styling

Manage the theme of your app, make your app responsive to screen sizes, or add padding.

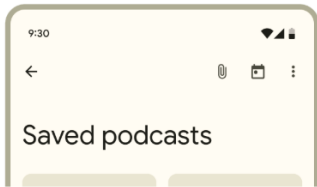
Text

Display and style text.

Basic widgets

A catalog of Flutter's basic widgets. Widgets to know before building your first Flutter app.

Widgets to know before building your first Flutter app.



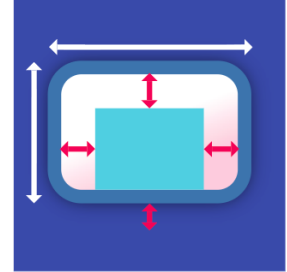
AppBar

Container that displays content and actions at the top of a screen.



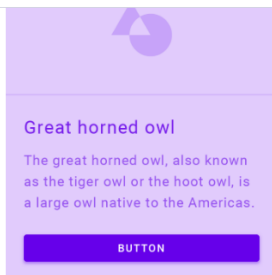
Column

Layout a list of child widgets in the vertical direction.



Container

A convenience widget that combines common painting, positioning, and sizing widgets.



ElevatedButton

A Material Design elevated button. A filled button whose material elevates when pressed.



FlutterLogo

The Flutter logo, in widget form. This widget respects the IconTheme.



Icon

A Material Design icon.



Image

A widget that displays an image.



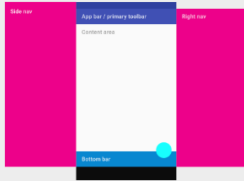
Placeholder

A widget that draws a box that represents where other widgets will one day be added.



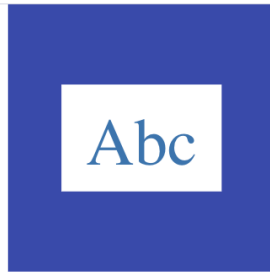
Row

Layout a list of child widgets in the horizontal direction.



Scaffold

Implements the basic Material Design visual layout structure. This class provides APIs for showing drawers, snack bars, and bottom sheets.



Text

A run of text with a single style.

Find more widgets in the [widget catalog](#).

2. Layouts

Link: <https://docs.flutter.dev/ui/layout>

2.1. Overview

What's the point?

- *Layouts in Flutter are built with widgets.*
- *Widgets are classes used to build UIs.*
- *Widgets are also used to build UI elements.*
- *Compose simple widgets to build complex widgets.*

The core of Flutter's layout mechanism is widgets. In Flutter, almost everything is a widget—even layout models are widgets. The images, icons, and text that you see in a Flutter app are all widgets. But things you don't see are also widgets, such as the rows, columns, and grids that arrange, constrain, and align the visible widgets. You create a layout by composing widgets to build more complex widgets.

2.2. Conceptual example

Bài tập 1. Vẽ widget tree

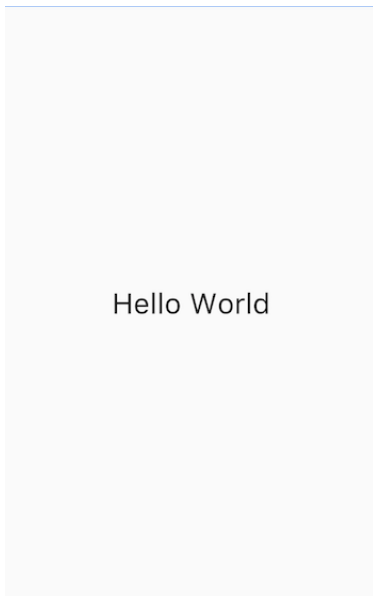
Hãy vẽ widget tree cho giao diện bên dưới:



2.3. Layout a widget

Bài tập 2. Bố cục cho một widget

Trong ví dụ bên dưới, hãy xác định đâu là layout widget, đâu là visible widget?

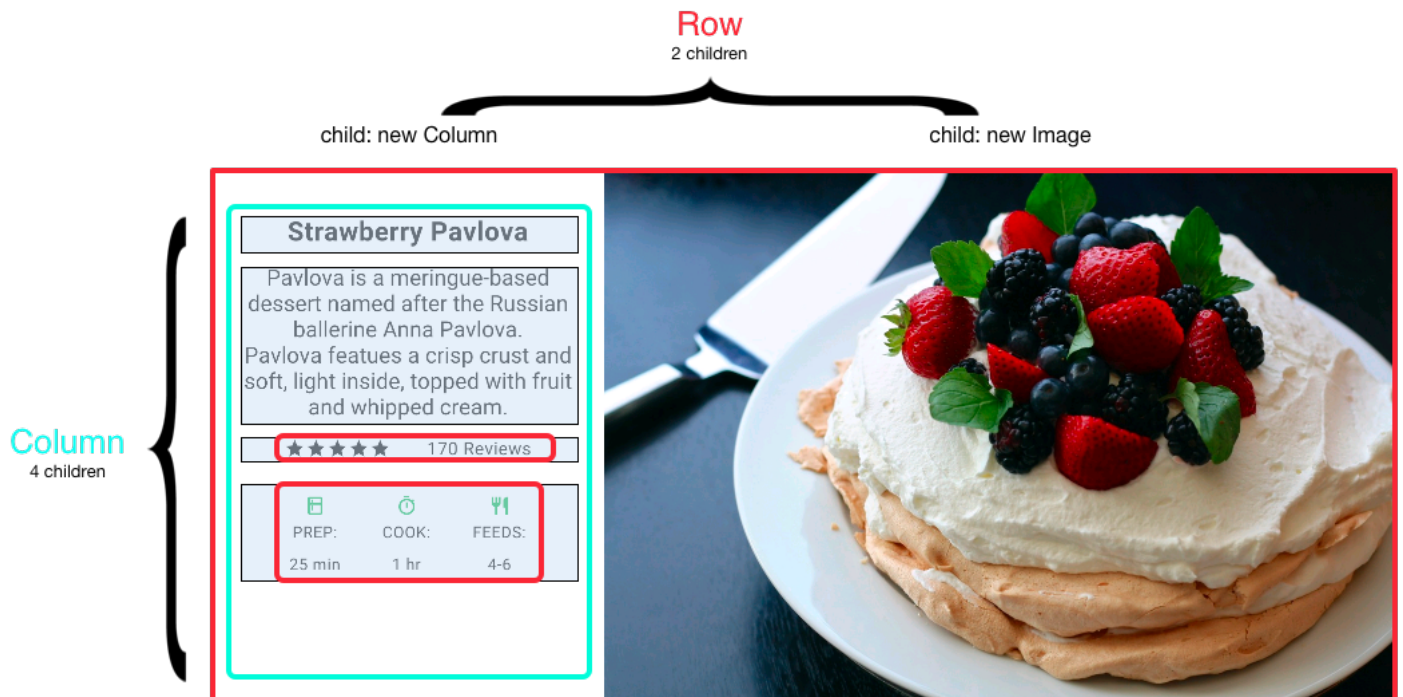


```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Container(  
      decoration: const BoxDecoration(color: Colors.white),  
      child: const Center(  
        child: Text(  
          'Hello World',  
          textDirection: TextDirection.ltr,  
          style: TextStyle(fontSize: 32, color: Colors.black87),  
        ),  
      ),  
    );  
  }  
}
```

2.4. Lay out multiple widgets vertically and horizontally

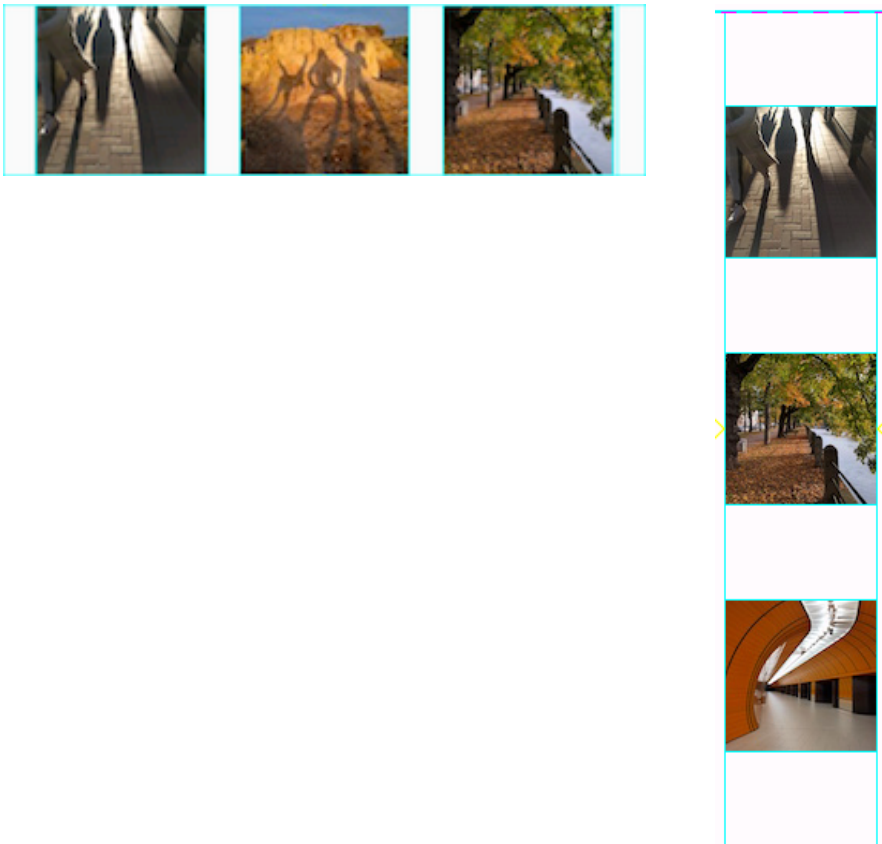
Bài tập 3. Vẽ cây widget

Hãy phân tích và vẽ cây widget cho giao diện sau:



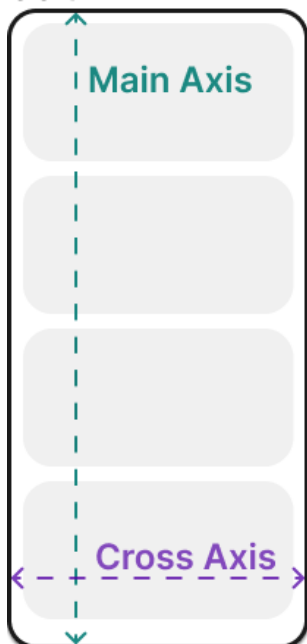
Bài tập 4. Aligning widgets

- a) Làm thế nào để sắp xếp các hình có khoảng cách đều nhau trong một Row hoặc Column, như hình:



- b) Hãy thử các giá trị khác nhau của: MainAxisAlignment & CrossAxisAlignment.

Column



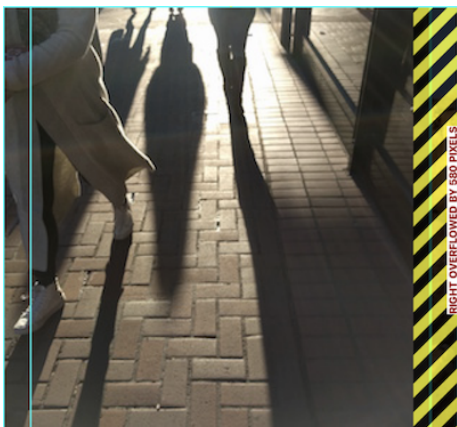
Row



Bài tập 5. Sizing widgets

Khi layout vượt quá kích thước thiết bị, một dải sọc vàng xuất hiện.

Để giải quyết, ta xử lý thế nào?



Bài tập 6. Packing widgets

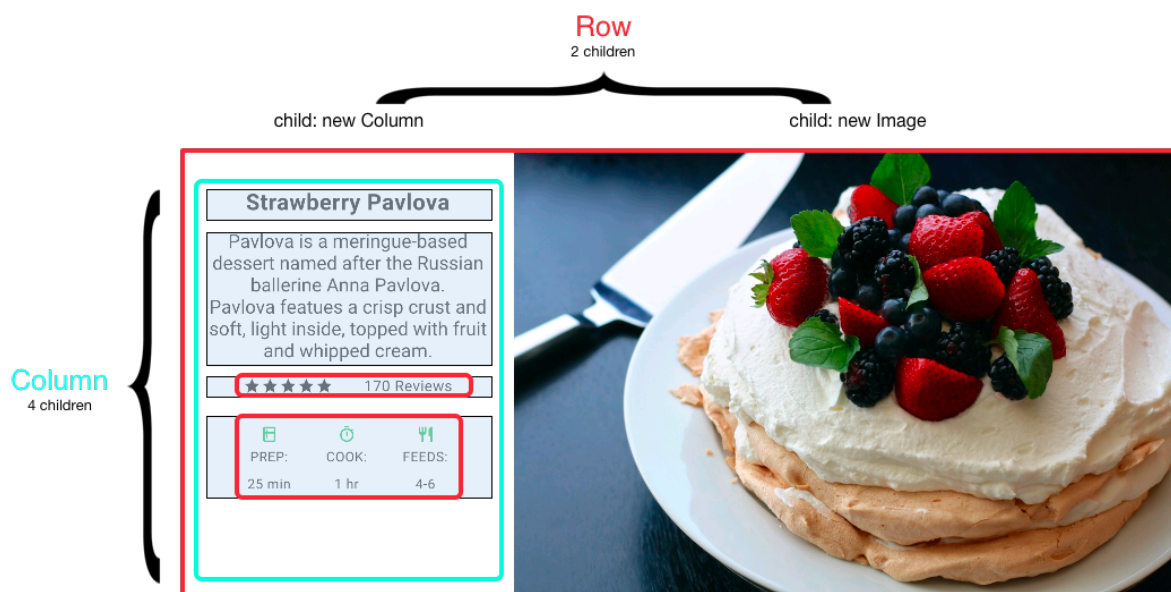
By default, a row or column occupies as much space along its main axis as possible, but if you want to pack the children closely together.

How to do that?



Bài tập 7. Nesting rows and columns

Hãy viết code xây dựng giao diện bên dưới:



2.5. Common layout widgets

2.6. Constraints

To fully understand Flutter's layout system, you need to learn how Flutter positions and sizes the components in a layout. For more information, see [Understanding constraints](#).

Link: <https://docs.flutter.dev/ui/layout/constraints>

3. Thực hành

Bài tập 8. Co giãn chiều cao Column theo con

🔗 Nguyên tắc của Column

Mặc định, Column sẽ chiếm hết chiều cao có sẵn của cha (nếu cha có ràng buộc chiều cao, ví dụ Scaffold.body → chiếm cả màn hình).

Nếu bạn muốn Column chỉ cao đúng bằng tổng chiều cao các con, thì cần bảo nó không mở rộng hết chiều cao cha.

✅ Cách làm

Có 2 cách phổ biến:

1. Dùng mainAxisAlignment: MainAxisAlignment.min

```
Column(  
  mainAxisAlignment: MainAxisAlignment.min, // chỉ cao bằng nội dung  
  children: [  
    Container(height: 50, color: Colors.red),  
    Container(height: 70, color: Colors.green),  
    Container(height: 30, color: Colors.blue),  
  ],  
)
```

🔗 Lúc này Column sẽ cao đúng $50 + 70 + 30 = 150$.

2. Bọc Column trong IntrinsicHeight

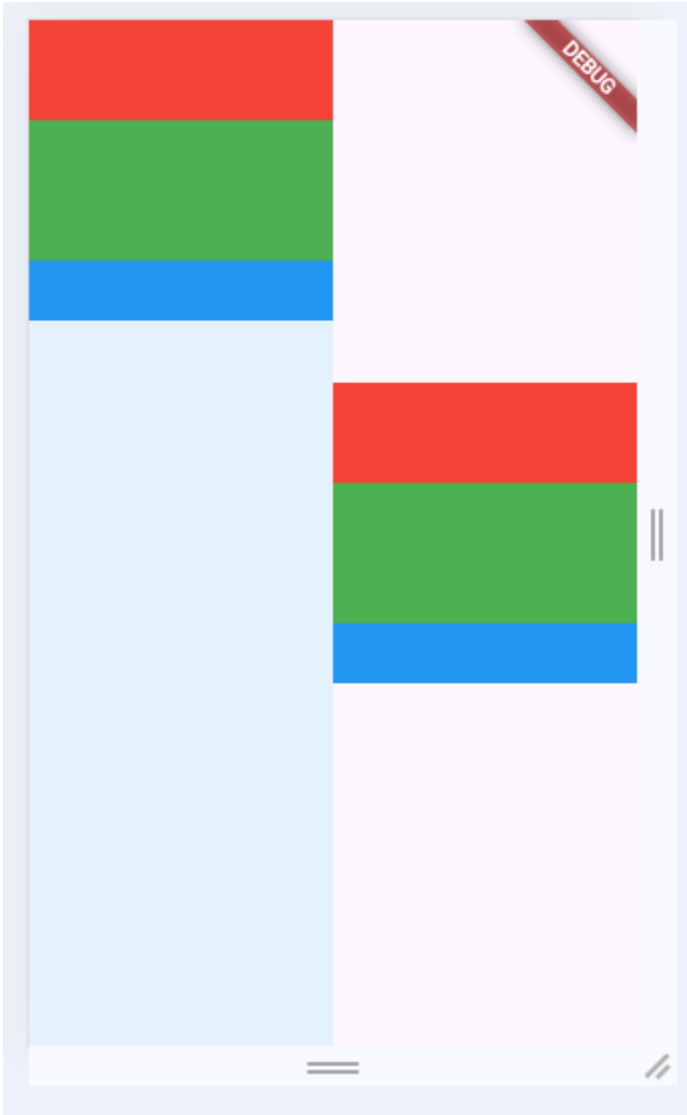
Nếu các con có chiều cao khác nhau phức tạp, bạn có thể dùng IntrinsicHeight để ép Column tính toán theo nội dung:

```
IntrinsicHeight(  
  child: Column(  
    children: [  
      Container(height: 80, color: Colors.orange),  
      Container(height: 40, color: Colors.purple),  
    ],  
  ),  
)
```

✦ Nhưng IntrinsicHeight tốn tài nguyên hơn → chỉ nên dùng khi thật sự cần.

🔗 So sánh

- `mainAxisSize: MainAxisSize.min`: nhanh, gọn, hiệu quả → dùng khi muốn Column co lại theo con.
- `IntrinsicHeight`: tính toán thêm → dùng khi cần layout phức tạp.



```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
```

```

    home: Scaffold(
      body: ExampleColumns(),
    ),
  );
}

class ExampleColumns extends StatelessWidget {
  const ExampleColumns({super.key});

  @override
  Widget build(BuildContext context) {
    return Row(
      children: [
        // Column mặc định: chiếm full height
        Expanded(
          child: Container(
            color: Colors.blue.shade50,
            child: Column(
              children: [
                Container(height: 50, color: Colors.red),
                Container(height: 70, color: Colors.green),
                Container(height: 30, color: Colors.blue),
              ],
            ),
          ),
        // Column co lại theo con: mainAxisAlignment.min
        Expanded(
          child: Container(
            color: Colors.yellow.shade50,
            child: Column(
              mainAxisAlignment: MainAxisAlignment.min, // ☞ chỉ cao bằng con
              children: [
                Container(height: 50, color: Colors.red),
                Container(height: 70, color: Colors.green),
                Container(height: 30, color: Colors.blue),
              ],
            ),
          ),
        ],
    );
  }
}

```

Bài tập 9. Tạo layout tổng hợp

Xây dựng Layout như hình bên dưới.

Tham khảo: <https://docs.flutter.dev/ui/layout/tutorial>

